

RNNs and LSTMs

CS221 - Natural Language Processing

Duy Dang Phu Anh Mai Quoc Bao Luong Huynh Gia Lac
Tran Nguyen Khai

University of Information Technology, VNU-HCM

Ngày 5 tháng 5 năm 2026

13.5 The LSTM

1. Sự quá tải của lớp ẩn

- Trọng số phải đảm nhận "nhiệm vụ kép": vừa xử lý đầu ra **hiện tại**, vừa lưu trữ thông tin cho **tương lai**.

2. Biến mất đạo hàm

- Khi huấn luyện qua chuỗi dài, đạo hàm bị triệt tiêu về 0, khiến mạng không thể cập nhật trọng số cho các mắt xích ở xa.

Hệ quả

Trong câu: *"The **flights** the airline was canceling **were** full"*

- RNN dễ quên từ "**flights**" (số nhiều) khi dự đoán đến từ "**were**", dẫn đến sai lệch ngữ pháp/ngữ nghĩa.

13.5 The LSTM

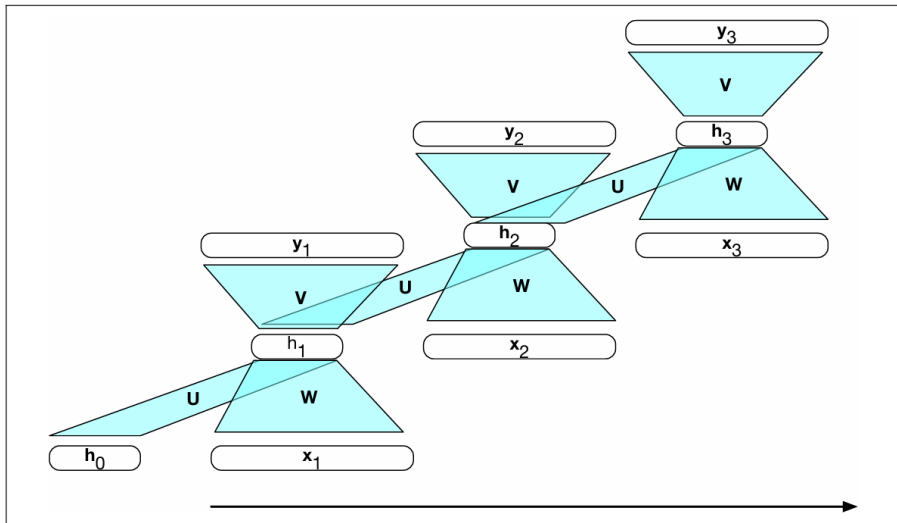


Figure 13.4 A simple recurrent neural network shown unrolled in time. Network layers are recalculated for each time step, while the weights U , V and W are shared across all time steps.

13.5 The LSTM

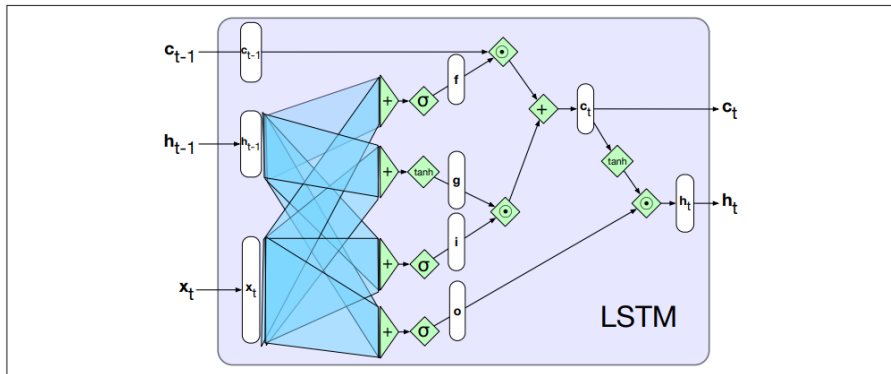
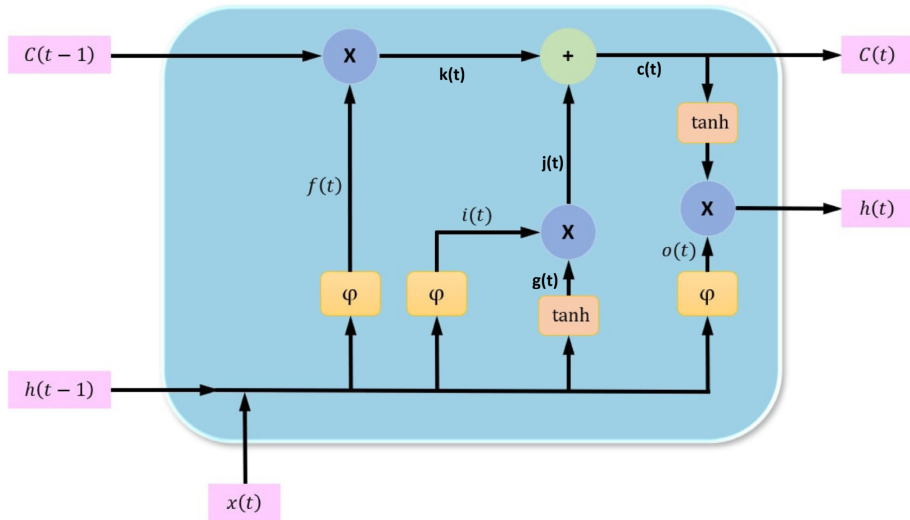
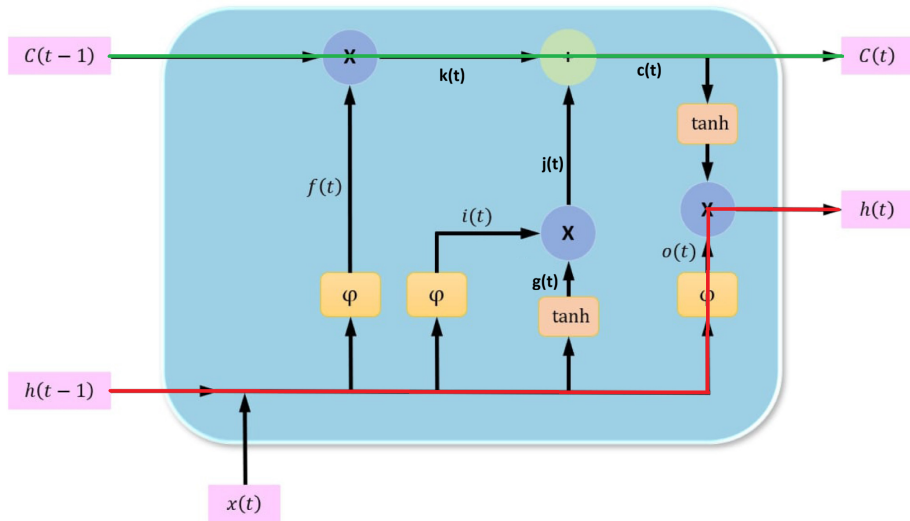


Figure 13.13 A single LSTM unit displayed as a computation graph. The inputs to each unit consists of the current input, x , the previous hidden state, h_{t-1} , and the previous context, c_{t-1} . The outputs are a new hidden state, h_t and an updated context, c_t .

13.5 The LSTM

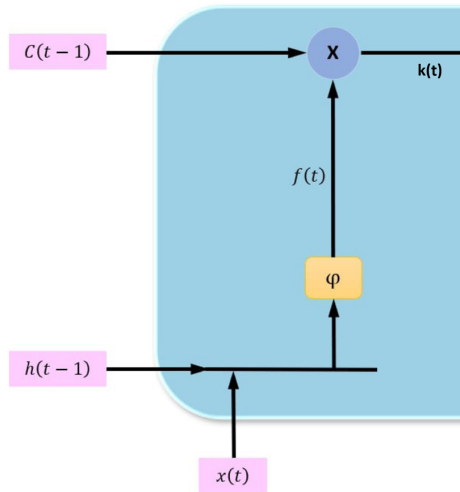


13.5 The LSTM



13.5 The LSTM - Forget Gate

- **Mục đích:** Xóa bỏ thông tin không còn cần thiết khỏi ngữ cảnh.
- **Công thức:**
$$f_t = \sigma(U_f h_{t-1} + W_f x_t) \quad (13.20)$$
$$k_t = c_{t-1} \odot f_t \quad (13.21)$$
- c_{t-1} : vector ngữ cảnh trước.
- k_t : ngữ cảnh sau khi lọc.



13.5 The LSTM - Add Gate

- Bước 1:** Tính thông tin tiềm năng:

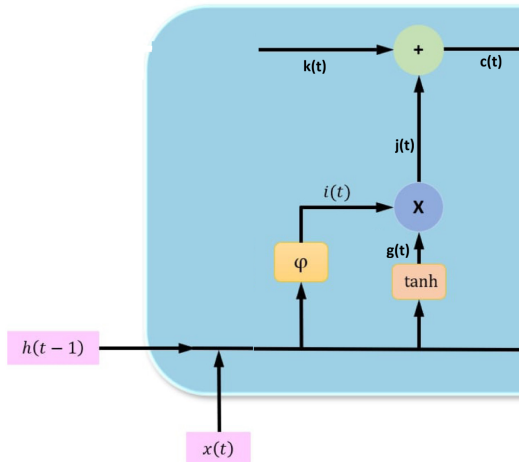
$$g_t = \tanh(U_g h_{t-1} + W_g x_t) \quad (13.22)$$

- Bước 2:** Tạo mặt nạ từ cổng vào:

$$i_t = \sigma(U_i h_{t-1} + W_i x_t) \quad (13.23)$$

$$j_t = g_t \odot i_t \quad (13.24)$$

- Cập nhật:** $c_t = j_t + k_t$



13.5 The LSTM - Output Gate

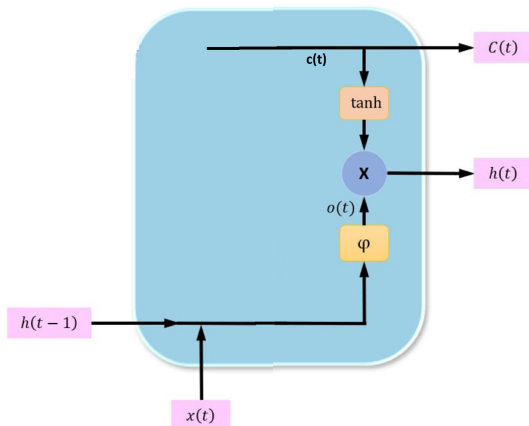
- Quyết định thông tin ngữ cảnh được xuất ra lớp ẩn.

- Công thức:**

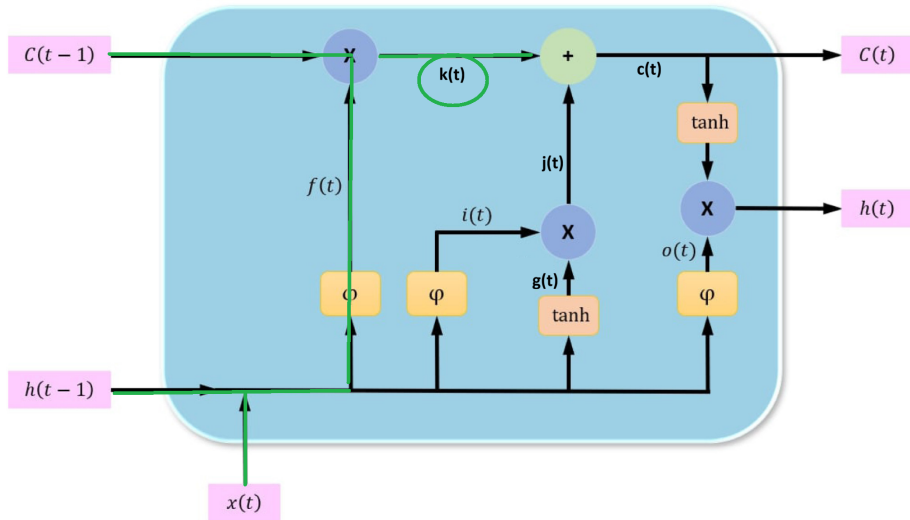
$$o_t = \sigma(U_o h_{t-1} + W_o x_t) \quad (13.26)$$

$$h_t = o_t \odot \tanh(c_t) \quad (13.27)$$

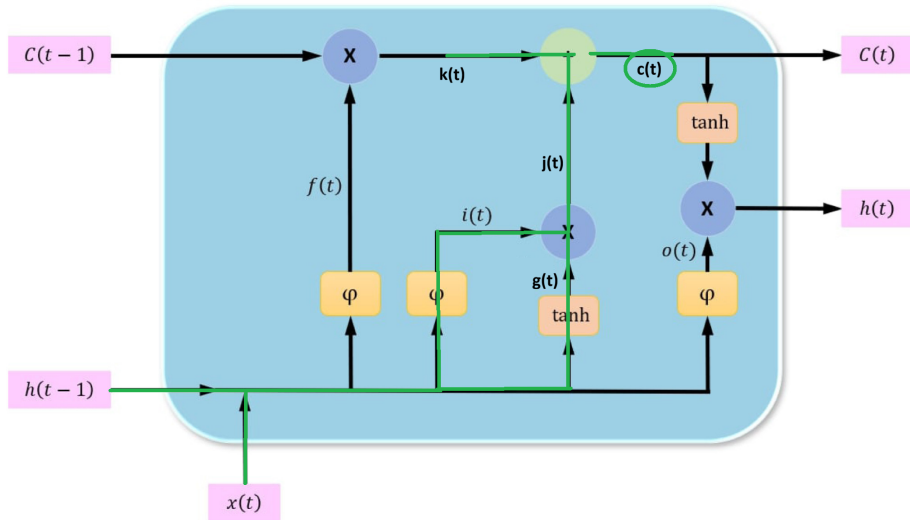
- h_t là đầu ra của đơn vị LSTM tại thời điểm hiện tại.



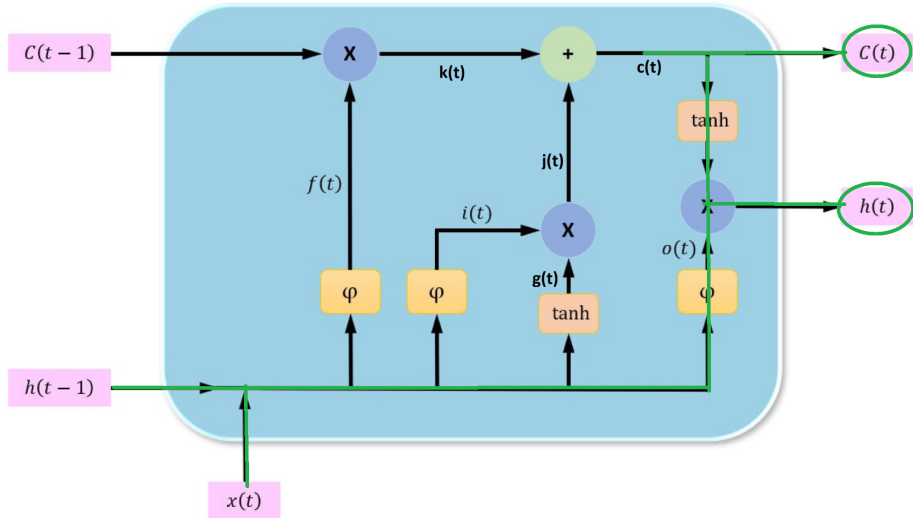
13.5 The LSTM



13.5 The LSTM



13.5 The LSTM



13.5.1 Gated Units, Layers and Networks

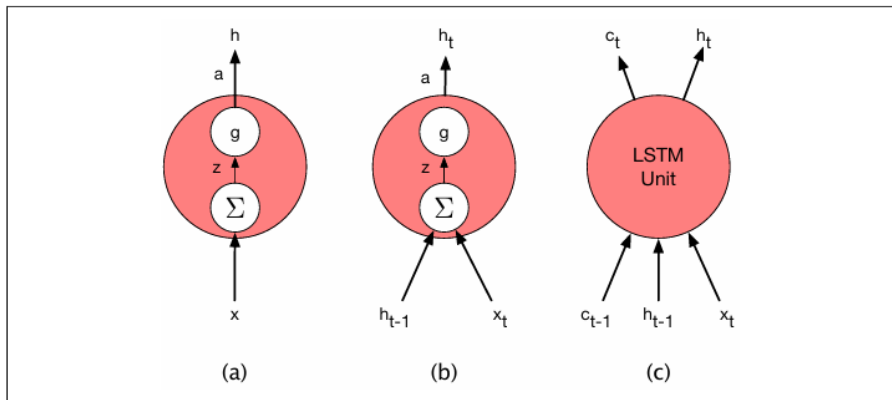


Figure 13.14 Basic neural units used in feedforward, simple recurrent networks (SRN), and long short-term memory (LSTM).

LSTM tuy phức tạp bên trong nhưng lại cực kỳ dễ sử dụng và linh hoạt bên ngoài nhờ tính đóng gói.

6 Vanishing and Exploding Gradients

The product of Jacobians in the BPTT gradient is the source of one of the most fundamental challenges in training RNNs. Each factor in the product is given by:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\sigma'(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{v}_t)) \cdot \mathbf{W}$$

The behavior of the full gradient depends on the spectral properties of these terms:

- **Vanishing gradients:** If $\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \right\| < 1$ consistently, then for large T , the product shrinks exponentially:

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \right\| \rightarrow 0$$

Early timesteps receive almost no gradient signal and fail to learn. The model effectively “forgets” information from the distant past.

- **Exploding gradients:** If $\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \right\| > 1$ consistently, the gradient grows exponentially:

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \right\| \rightarrow \infty$$

Parameters become unstable and training diverges. Gradient clipping can mitigate this, but it does not solve the underlying problem.

This explains why **vanilla RNNs struggle with long sequences**: the hidden states cannot reliably retain useful information across many timesteps because the gradient either vanishes or explodes during backpropagation.

Gradient trong LSTM

The cell state update equation $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ is the critical innovation: the additive structure (rather than multiplicative) allows gradients to flow through the cell state without the compounding decay that plagues vanilla RNNs. In essence, **LSTMs solve the same problem that ResNets solved—but in time rather than depth.**

Sự tương đồng về dòng chảy Gradient

- **ResNet:** Kết nối tắt $H(x) = F(x) + x$ tạo ánh xạ đồng nhất, giúp Gradient truyền qua các tầng mà không bị suy giảm.
- **LSTM:** *Cell State* đóng vai trò "đường cao tốc". Khi $f_t \approx 1$, phép cộng giúp bảo toàn Gradient xuyên suốt các bước thời gian dài.

Gradient trong LSTM

For highway layers, we use the transform gate defined as $T(\mathbf{x}) = \sigma(\mathbf{W}_T^T \mathbf{x} + \mathbf{b}_T)$, where $\mathbf{W}_T$ is the weight matrix and $\mathbf{b}_T$ the bias vector for the transform gates. This suggests a simple initialization scheme which is independent of the nature of H : $\mathbf{b}_T$ can be initialized with a negative value (e.g. -1, -3 etc.) such that the network is initially biased towards *carry* behavior. This scheme is strongly inspired by the proposal of Gers et al. (1999) to initially bias the gates in a Long Short-Term Memory recurrent network to help bridge long-term temporal dependencies early in learning. Note that $\sigma(x) \in (0, 1), \forall x \in \mathbb{R}$, so the conditions in Equation (4) can never be exactly true.

Paper Highway Networks

Concurrent with our work, “highway networks” [42, 43] present shortcut connections with gating functions [15]. These gates are data-dependent and have parameters, in contrast to our identity shortcuts that are parameter-free. When a gated shortcut is “closed” (approaching zero), the layers in highway networks represent *non-residual* functions. On the contrary, our formulation always learns residual functions; our identity shortcuts are never closed, and all information is always passed through, with additional residual functions to be learned. In addition, high-

Paper ResNets

Tại sao lại là h_{t-1} ?

Cell state and hidden state in LSTMs

An LSTM maintains two separate vectors that flow through the network: the cell state and the hidden state.

The **cell state** acts as long-term storage. It carries information across timesteps with controlled modifications at each step. When processing our sequence, the cell state might store "weather-related noun appeared" after seeing "clouds" and preserve that context through "are", until "dark" reinforces the pattern.

The **hidden state** is the cell's output at each timestep. It feeds into predictions and into the next timestep's computations. After processing "dark," the hidden state encodes something like "adjective describing weather conditions" that a prediction layer can use.

S